

GEN AI & PROMPT ENGG. | CAPSTONE PROJECT 2026

MediClarity

AI-Powered Medical Report Explainer

NAME	Sarthak Pattnaik
ROLL NUMBER	23052666
BATCH	Gen AI & Prompt Engineering – Batch 1
SUBMITTED TO	Sachin Lohar
DATE	September 2026

1. Problem Statement

Medical reports are an essential part of healthcare, providing information about a person's health through laboratory investigations, diagnostic tests, clinical observations, and medical measurements. However, these reports frequently contain technical terminology, medical abbreviations, numerical values, and reference ranges that may be difficult for individuals without a medical background to understand.

Patients and caregivers may find it challenging to interpret laboratory results, understand unfamiliar medical terms, or identify which findings they should discuss with a healthcare professional. Searching for individual terms online can also produce confusing or contextually inappropriate information.

MediClarity addresses this problem by providing a web-based platform that processes uploaded medical reports and generates AI-assisted explanations in understandable language. The system aims to:

- Simplify medical terminology and complex textual information.
- Extract readable text from digital and scanned medical documents.
- Organize relevant clinical information into structured categories.
- Generate understandable explanations using Generative AI.
- Provide authenticated users with access to their previously processed reports.

MediClarity is designed as an informational assistant. It does not independently diagnose diseases, recommend treatments, or replace qualified medical professionals.

2. Objectives and Scope

2.1 Objectives

The main objectives of MediClarity are:

1. To develop a user-friendly web application for uploading and understanding medical reports.
2. To implement text extraction from PDF documents using PyMuPDF.
3. To implement OCR-based text extraction from scanned documents and images using Tesseract.
4. To integrate Generative AI APIs for clinical-context extraction and plain-language explanation generation.
5. To implement authentication and user-specific access to stored medical reports.
6. To store report information and generated explanations using Supabase services.
7. To deploy the frontend and backend on cloud platforms and make the application accessible through a live URL.
8. To test the complete application workflow, including document upload, processing, explanation generation, and report retrieval.

2.2 Scope of the Project

The project covers the following functional areas:

User authentication: Users can authenticate and access their own report workspace.

Medical report upload: The application accepts PDF, PNG, JPG, and JPEG files.

Document processing: Text is extracted from PDFs and image-based reports.

AI-assisted understanding: Extracted content is processed using language models to structure clinical information and generate explanations.

Report storage and retrieval: Report information and explanations are stored and made accessible to the authenticated user.

Cloud deployment: The frontend and backend are deployed separately, allowing users to access the application through the internet.

3. Solution and Features

3.1 Overview of the Solution

MediClarity follows a client-server architecture consisting of a Next.js frontend and a FastAPI backend.

The frontend provides the interface for authentication, uploading reports, viewing saved reports, and reading explanations. The backend manages authentication-related requests, document processing, OCR, AI integration, database operations, and access control. The application uses Supabase PostgreSQL for structured data storage and Supabase Storage for uploaded medical documents. Groq and the configured Gemini integration provide AI-related functionality.

3.2 Major Features

3.2.1 User Authentication

The application provides authentication functionality to ensure that users can access their own report workspace. The frontend maintains authentication state through an authentication context and uses an access token for authenticated API requests. The backend performs authorization and ownership checks before returning protected report information.

3.2.2 Medical Report Upload

Users can upload medical reports through the web interface. The supported file formats include: PDF, PNG, JPG and JPEG. The frontend submits uploaded files to the backend using a multipart/form-data request. The backend validates and processes the uploaded file according to its format.

3.2.3 PDF Text Extraction

PDF documents are processed using PyMuPDF. PyMuPDF extracts text from the document's existing text layer, allowing digitally generated medical reports to be processed without requiring OCR when readable text is available.

3.2.4 Image-Based Text Extraction

Scanned medical documents and image-based reports are processed using Optical Character Recognition. The application uses the Python package pytesseract, which communicates with the Tesseract OCR engine. This allows the system to extract text from supported image formats such as PNG, JPG, and JPEG.

3.2.5 Clinical Context Extraction

The extracted text is processed using a Generative AI model to identify and organize relevant clinical information. The structured information includes medical history, symptoms, medications, clinical

findings, recommendations, relevant context, and context notes. A Pydantic schema is used to define the expected structure of the clinical-context output and support validation.

3.2.6 AI-Generated Explanation

The application uses generative AI to transform medical-report information into a more understandable explanation. The generated explanation is intended to help users understand the textual content of their report and familiarize themselves with medical terminology.

The explanation is stored and can be retrieved through the report-specific API.

3.2.7 Report History and Retrieval

Authenticated users can access their saved report records and retrieve the corresponding explanations. The backend checks report ownership before providing protected report information. This helps ensure that users cannot access reports belonging to other accounts through protected API endpoints.

3.2.8 Cloud Deployment

The frontend is deployed on Vercel, while the FastAPI backend is deployed on Render using Docker. Docker is used to package the backend and its dependencies, including the Tesseract OCR executable required for image processing.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
Next.js	Frontend framework	Implements the web application using the App Router.
FastAPI	Backend framework	Provides REST API endpoints and request validation.
Supabase PostgreSQL	Database	Stores structured application and report-related data.
Supabase Storage	File storage	Stores uploaded medical reports in a private bucket.
PyMuPDF	PDF processing	Extracts text from PDF documents.
TesseractOCR	OCR engine	Recognizes text in scanned documents and images.
Groq API	Generative AI	Supports clinical-context extraction and AI text generation.

5. Knowledge Base and Data

5.1 Data Source

MediClarity primarily processes medical reports uploaded by users rather than relying on a fixed, curated medical knowledge base. The uploaded report acts as the primary source of information for the extraction and explanation pipeline. The application converts the uploaded file into extracted text and then uses Generative AI to organize and explain the information. The system's output is therefore dependent on the content and quality of the original report, the text extraction process, and the AI model's response.

5.2 Privacy and Access Control

Medical reports may contain sensitive personal information. Therefore, access control and storage security are important parts of the application.

The following measures are implemented:

- Uploaded files are stored in a private Supabase Storage bucket.
- File paths are organized under the relevant user's identifier.
- Protected backend routes check report ownership before returning report information.
- Authentication is required for access to protected report data.

These controls help restrict access to user-specific information. The application should not be considered formally security-audited or clinically certified.

6. System Architecture

6.1 System Architecture Diagram



6.2 Components of the Architecture

Frontend Layer

The frontend is developed using Next.js and React. It provides the user interface for authentication, report uploading, report history, and explanation display. The frontend sends authenticated API requests to the backend and presents the returned information to the user.

Backend Layer

FastAPI serves as the central processing layer. It receives requests from the frontend and coordinates authentication, file handling, text extraction, AI integration, database operations, and report retrieval. It also enforces authorization and report ownership checks for protected resources.

Document Processing Layer

The document-processing layer extracts text from uploaded files. PyMuPDF is used for PDF documents, while Tesseract OCR is used for image-based reports. The extracted text is passed to the AI processing components.

AI Processing Layer

The AI processing layer uses Groq and the configured Gemini integration to process extracted medical information. The clinical-context extraction component structures relevant information, while the explanation-generation component produces a simplified explanation for the user.

Database and Storage Layer

Supabase PostgreSQL stores structured application information. Supabase Storage stores the original uploaded documents in a private bucket. The backend connects these stored files and report records to the authenticated user.

7. Implementation Highlights

7.1 Medical Report Upload

The report upload functionality is implemented using a FastAPI endpoint that receives files from the frontend. The frontend creates a FormData object, appends the selected file, and sends it to the backend through an Axios request. The backend processes the file based on its extension and content type.

7.2 PDF Text Extraction Using PyMuPDF

PyMuPDF is used to extract text from PDF documents. When a PDF contains a readable text layer, the backend can extract its text directly without performing OCR on every page. The extracted text is then used as input for the subsequent AI processing stages. This approach is suitable for digitally generated reports containing selectable text. For scanned PDFs containing only page images, OCR is required to recognize the text.

7.3 Image Text Extraction Using Tesseract OCR

Tesseract OCR is used to extract text from image-based medical reports. The backend uses the Python package pytesseract to communicate with the Tesseract OCR executable.

7.4 Clinical Context Extraction Using Generative AI

The extracted text is processed using a Groq language model to identify and organize clinically relevant information. A Pydantic model defines the expected structure of the clinical-context output. The model contains fields such as medical history, symptoms, medications, clinical findings, recommendations, relevant context, and context notes. Structured output is used to make the AI response more predictable and easier for the backend to validate. The application also has a configured alternative AI integration through Gemini.

7.5 AI-Generated Medical Explanation

After the clinical information is extracted, the application generates a plain-language explanation of the medical report. The explanation is intended to make the report's contents more understandable to users who may not be familiar with medical terminology. The generated explanation is associated with the relevant report and stored for subsequent retrieval. The application exposes a report-specific API endpoint for retrieving saved explanations.

7.6 Authentication and Report Ownership

The application uses authentication to restrict access to protected features. The frontend maintains authentication state through an authentication context and restores the access token when the application is loaded. The backend performs ownership checks before returning protected report data. This means that access to a report depends on the authenticated user's identity and ownership of the corresponding report. The backend remains responsible for enforcing access restrictions, rather than relying only on frontend navigation or visibility.

7.7 Deployment and Environment Configuration

The frontend and backend are deployed separately. The frontend is hosted on Vercel, while the backend is hosted on Render using Docker. The frontend communicates with the deployed backend through its configured API base URL. Cross-Origin Resource Sharing (CORS) is configured to allow requests from the deployed frontend origin. Docker packages the backend dependencies and system-level requirements, including Tesseract OCR, making the hosted environment more consistent with the expected runtime.

8. Testing and Results

Test ID	Test Case	Input / Condition	Expected Result	Status
TC-01	Open deployed application	Live Vercel URL	Application loads successfully	Passed
TC-02	User authentication	Valid user credentials	User can access the authenticated workspace	Passed
TC-03	PDF upload	Supported PDF file	File is accepted and processed	Passed
TC-04	PNG upload	PNG medical report	OCR extracts text and upload workflow completes	Passed
TC-05	JPG upload	JPG medical report	Image is accepted and processed	Passed
TC-06	JPEG upload	JPEG medical report	Image is accepted and processed	Passed
TC-07	Explanation display	Successfully processed report	Generated explanation is displayed	Passed
TC-08	Explanation retrieval	Saved report	Explanation can be retrieved through the application	Passed
TC-09	Report history	Authenticated user with saved reports	User can access their saved reports	Passed
TC-10	Report ownership validation	Request for a report belonging to another user	Unauthorized access is denied	Passed

9. Evaluation and Metrics

Evaluation Criterion	Evaluation Method	Result
Supported file formats	Manual upload testing	PDF, PNG, JPG, and JPEG reported working
OCR processing	Image upload and processing tests	Image OCR workflow reported working
End-to-end functionality	Live application testing	Developer reports tested workflows ran smoothly
Explanation generation	Testing processed reports	Explanation generation and display reported working
Report retrieval	Accessing saved reports and explanations	Retrieval functionality reported working
Cloud deployment	Testing deployed frontend and backend	Both services deployed and accessible

10. Screenshots

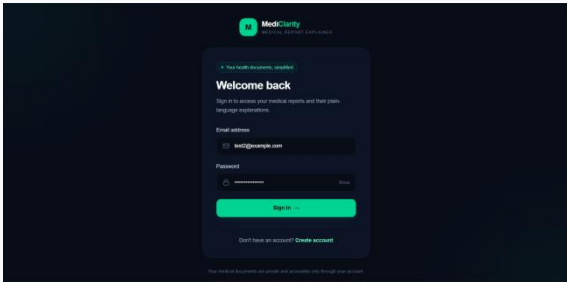


Figure 10.1: Login interface of the MediClarity application.

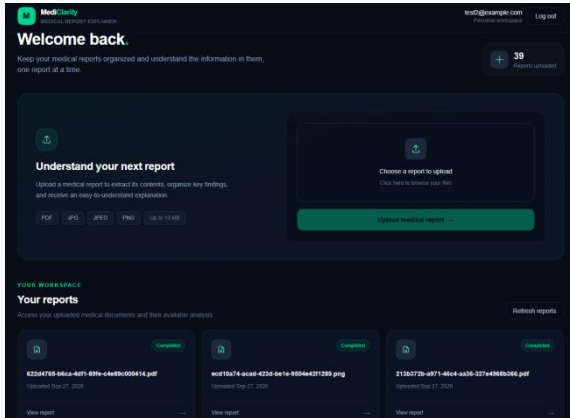


Figure 10.2: Dashboard displaying the user's report workspace.

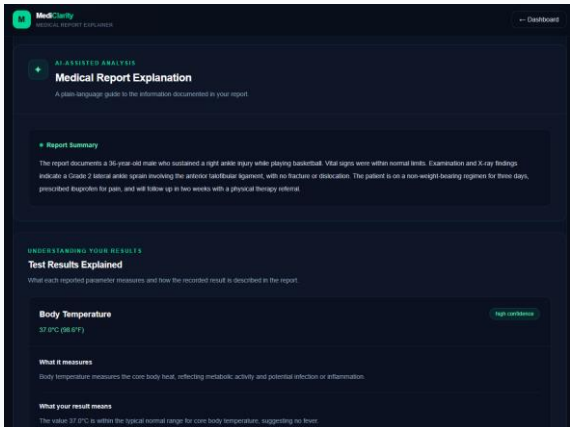


Figure 10.3: AI-generated plain-language explanation of an uploaded medical report.

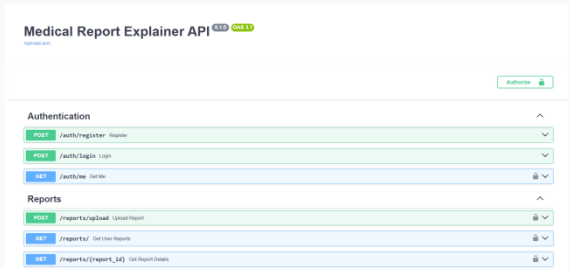


Figure 10.4: FastAPI Swagger documentation for the MediClarity backend.

11. Challenges and Learnings

During the development and deployment of MediClarity, several technical challenges were encountered. Addressing these challenges provided practical experience in full-stack development, AI integration, document processing, and cloud deployment.

11.1 OCR Dependency in the Production Environment

One of the main challenges occurred when image uploads worked locally but failed on the initial deployed backend. The hosted backend returned an error indicating that Tesseract was not installed or available in the system PATH. The issue arose because the Python package pytesseract requires the underlying Tesseract executable to be installed separately. The problem was resolved by deploying the backend using Docker and installing Tesseract inside the container. This demonstrated the importance of managing both Python dependencies and operating-system dependencies during deployment.

11.2 Handling Long AI-Generated Responses

During clinical-context extraction, a long medical case caused the Groq API to return a structured-output validation error. The response reached its completion-token limit before all required fields were generated. This highlighted the need for:

- Appropriate output-token limits.
- Structured output validation.
- Handling incomplete or invalid model responses.
- Explicit error handling and fallback behavior.
- Managing the size of extracted clinical information.

The experience demonstrated that AI-generated output must be validated before being used by downstream application components.

11.3 Key Learnings

The major learnings from the project include:

1. Integrating generative AI into a real application requires reliable input processing, output validation, and error handling.
2. OCR applications may depend on system-level executables in addition to Python libraries.
3. Docker helps package backend dependencies and create a consistent deployment environment.
4. Authentication, authorization, and private storage are important when working with user-specific documents.
5. Frontend and backend deployments require careful API URL, CORS, and environment-variable configuration.
6. Functional testing of the complete deployed workflow is necessary to identify problems that may not occur in local development.

12. Future Improvements

Although MediClarity implements the primary document-processing and explanation workflow, several improvements can be considered for future development.

12.1 Improved OCR Accuracy

The OCR pipeline can be enhanced to handle low-resolution images, rotated documents, skewed pages, complex tables, and additional languages. Image preprocessing techniques such as denoising, thresholding, and deskewing could be introduced to improve text recognition.

12.2 Clinical Evaluation

A formal evaluation process can be developed using a de-identified dataset of medical reports. The generated explanations could be reviewed by qualified healthcare professionals using a defined evaluation rubric covering readability, faithfulness to the source document, completeness, and potentially harmful interpretations. This would provide more meaningful evidence about the quality and limitations of the generated explanations.

12.3 Report Export Functionality

A future version could allow users to download their explanations as PDF documents. The exported document could include the original report's relevant values, the generated explanation, and appropriate informational disclaimers.

12.4 User-Controlled Data Management

Additional functionality could allow users to delete their uploaded reports and associated stored data. A documented data-retention policy and clear controls for managing stored medical documents could further improve transparency and privacy.

13. Conclusion

MediClarity demonstrates the practical application of Generative AI and full-stack web development to the problem of understanding medical reports.

The project combines a Next.js frontend, FastAPI backend, Supabase PostgreSQL database, private file storage, PDF text extraction, Tesseract OCR, and generative AI services to provide an integrated medical-report explanation workflow.

The application allows authenticated users to upload medical reports in PDF and common image formats, process the documents, generate plain-language explanations, and retrieve saved report information.

The project also incorporates authentication and report ownership checks to protect user-specific data.

The application has been deployed with the frontend hosted on Vercel and the backend hosted on Render using Docker. A production OCR dependency issue was identified and resolved by including the Tesseract executable in the Docker image. The developer reports that the live application was tested across its functionality and that the tested workflows operated smoothly.

The project provided valuable practical experience in frontend and backend development, REST API integration, OCR-based document processing, structured LLM output, database and storage integration, authentication, and cloud deployment.

MediClarity illustrates how Generative AI can improve access to complex technical information when combined with appropriate software engineering practices. However, it remains an informational prototype and requires formal clinical, privacy, and security evaluation before being used in clinical decision-making.

14. References and Links

Item	URL
MediClarity Live Application	https://medical-report-explainer-lake.vercel.app
MediClarity Backend	https://medical-report-explainer-backend.onrender.com
FastAPI Swagger Documentation	https://medical-report-explainer-backend.onrender.com/docs
MediClarity GitHub Repository	https://github.com/Sarthak-Pattnaik/medical-report-explainer
FastAPI Documentation	https://fastapi.tiangolo.com/
Next.js Documentation	https://nextjs.org/docs
Supabase Documentation	https://supabase.com/docs
PyMuPDF Documentation	https://pymupdf.readthedocs.io/
pytesseract Documentation	https://pypi.org/project/pytesseract/
Tesseract OCR Repository	https://github.com/tesseract-ocr/tesseract
Groq API Documentation	https://console.groq.com/docs
Google Gemini API Documentation	https://ai.google.dev/gemini-api/docs
Render Documentation	https://render.com/docs
Vercel Documentation	https://vercel.com/docs